



INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo “ESCOM”

Valencia Villanueva Adrian

2026630144

Investigación Bases de Datos

Sistemas de Bases de Datos

Dato, información y sistema de base de datos

Un **dato** es un hecho crudo sin interpretar (una cadena de caracteres, un número), se convierte en **información** cuando se procesa y contextualiza para tener significado para quien lo consume. Elmasri y Navathe (2016) definen una base de datos como una colección de datos relacionados con un significado implícito, organizada para representar aspectos del mundo real y construida con un propósito específico, enfatizan que los datos son lógicamente coherentes entre sí, no una colección arbitraria. Silberschatz et al. (2019) llegan a una definición funcionalmente equivalente pero centrada en el software, un sistema de base de datos es la combinación de los datos y del conjunto de programas que permiten acceder a ellos y modificarlos el **DBMS** (Database Management System), cuyo objetivo explícito es proveer una forma conveniente y eficiente de almacenar y recuperar información. La primera definición es ontológica (qué es una base de datos), la segunda es operacional (qué hace un DBMS con ella), ambas concuerdan en que el valor de un sistema de base de datos no está en el almacenamiento en sí, sino en la disponibilidad controlada y eficiente de esos datos para múltiples consumidores concurrentes.

Deficiencias del enfoque de archivos planos

Antes de la adopción de DBMS, cada programa de aplicación mantenía sus propios archivos de datos privados. Elmasri y Navathe (2016) y Date (2004) documentan de forma consistente las deficiencias estructurales de ese enfoque:

- **Redundancia e inconsistencia:** El mismo dato (ej. la dirección de un cliente) se duplica en archivos de distintos programas, una actualización parcial deja copias desincronizadas.
- **Dificultad de acceso a los datos:** Cualquier consulta no anticipada por el diseño original del archivo requiere escribir un nuevo programa.
- **Aislamiento de datos:** Los datos están dispersos en archivos con formatos distintos, dificultando escribir aplicaciones que crucen información de varias fuentes.

●**Problemas de integridad:** Las restricciones de negocio (ej. el saldo no puede ser negativo) quedan embebidas en el código de la aplicación en lugar de centralizadas, por lo que cada programa nuevo debe reimplementarlas.

●**Anomalías de actualización:** Inserción (no se puede registrar un dato sin otro dato relacionado que aún no existe), eliminación (borrar un registro elimina accidentalmente información que se quería conservar) y modificación (actualizar un valor duplicado en múltiples lugares).

●**Fallas de atomicidad y control de concurrencia deficiente:** Sin un gestor de transacciones, una falla a mitad de una operación de múltiples pasos deja los datos en un estado parcial e inconsistente, sin control de concurrencia, dos procesos que leen-modifican-escriben el mismo archivo en paralelo pueden perder actualizaciones (lost update).

Arquitectura ANSI-SPARC

La arquitectura de tres niveles (propuesta por el comité ANSI/X3/SPARC y descrita en detalle por Silberschatz et al., 2019, y Elmasri y Navathe, 2016) separa la representación de los datos en tres capas para lograr independencia entre cómo se almacenan físicamente y cómo los perciben los usuarios:

●**Nivel físico (interno):** Describe cómo se almacenan realmente los datos estructuras de archivos, índices, compresión, ubicación en disco.

●**Nivel conceptual (lógico):** Describe qué datos se almacenan y qué relaciones existen entre ellos, sin detalles de implementación física, es el nivel del modelo entidad-relación o del esquema relacional completo.

●**Nivel externo (de vistas):** Describe la porción del esquema conceptual relevante para un grupo de usuarios o aplicación particular (una vista), ocultando el resto.

Los **mappings** (conceptual/interno y externo/conceptual) son las funciones de traducción que el DBMS mantiene entre niveles, su existencia es precisamente lo que permite modificar un nivel sin romper los superiores.

Independencia de datos y sublenguajes

La **independencia física de datos** es la capacidad de modificar el esquema físico (ej. agregar un índice o cambiar el algoritmo de compresión) sin alterar el esquema conceptual ni las aplicaciones. La **independencia lógica de datos** es la capacidad de modificar el esquema conceptual (ej. agregar un nuevo atributo) sin forzar cambios en todos los programas que usan vistas externas que no dependen de ese atributo. Date (2004) señala que la independencia lógica es la más difícil de lograr en la práctica, porque cambios conceptuales tienden a propagarse hacia las vistas con mayor facilidad que los cambios puramente físicos.

Los sublenguajes de datos se dividen taxonómicamente en:

●**DDL** (Data Definition Language): Define y modifica esquemas CREATE, ALTER, DROP.

●**DML** (Data Manipulation Language): Consulta y modifica instancias de datos SELECT, INSERT, UPDATE, DELETE.

●**DCL** (Data Control Language): Gestiona permisos GRANT, REVOKE.

●**TCL** (Transaction Control Language): Delimita unidades atómicas de trabajo COMMIT, ROLLBACK, SAVEPOINT.

Diagrama modular de los componentes internos de un DBMS (Silberschatz et al., 2019; Ramakrishnan y Gehrke, 2003):



Taxonomía de motores por caso de uso a escala

●**Relacionales (RDBMS):** PostgreSQL, MySQL, Oracle. Óptimos cuando la integridad transaccional (ACID) y las relaciones estructuradas entre entidades son el requisito dominante como en: sistemas financieros, ERP.

●**Orientadas a documentos:** MongoDB, Couchbase. Esquema flexible por documento (típicamente JSON/BSON), adecuadas para catálogos de producto con atributos heterogéneos o datos que evolucionan rápido en forma.

●**Clave-valor:** Redis, DynamoDB. Latencia mínima para operaciones de lectura/escritura simples por clave, casos de uso típicos: sesiones de usuario, cachés, contadores de alta frecuencia.

●**Familias de columnas:** Cassandra, HBase. Optimizadas para escritura masiva distribuida y consultas por rango sobre grandes volúmenes como en: series de eventos, telemetría a escala de petabytes.

●**Grafos:** Neo4j, Amazon Neptune. Modelan relaciones como ciudadanos de primera clase, útiles para detección de fraude, motores de recomendación y análisis de redes sociales, donde la operación dominante es el recorrido de relaciones (traversal) más que el filtrado tabular.

●**Series temporales:** InfluxDB, TimescaleDB. Indexación y compresión especializadas por marca de tiempo; monitoreo de infraestructura, IoT.

●**Geoespaciales:** PostGIS (extensión de PostgreSQL). Tipos e índices espaciales (R-tree) para consultas de proximidad y contención geográfica.

●**Vectoriales:** Milvus, Pinecone, pgvector. Indexación para búsqueda por similitud aproximada (ANN) sobre embeddings de alta dimensionalidad.

Criterios de clasificación de DBMS

●**Modelo lógico de datos:** Relacional, documental, grafo, clave-valor, columnar (determina la forma en que se expresan las relaciones entre datos).

●**Arquitectura de concurrencia:** Monousuario vs Multiusuario, dentro de multiusuario, control por bloqueos (locking) vs control multiversión (MVCC), este último es el usado por PostgreSQL para permitir lecturas no bloqueantes.

●**Distribución geográfica:** Centralizada, distribuida homogénea (mismo motor en todos los nodos) o distribuida heterogénea (federación de motores distintos).

●**Propósito:** Propósito general vs Especializado (series temporales, vectorial, geoespacial).

●**Esquema de licenciamiento:** Propietario (Oracle), open-source con núcleo libre (PostgreSQL, MySQL Community) u open-core (con funcionalidades empresariales cerradas).

Almacenamiento y recuperación en pipelines de Inteligencia Artificial

Los pipelines de IA introducen patrones de acceso a datos que los DBMS clásicos no anticipaban en su diseño original:

●**Datasets de entrenamiento:** Requieren lectura secuencial masiva y reproducible (versionado de dataset, no solo de fila) más que transacciones puntuales, de ahí el auge de formatos columnares (Parquet) y data lakes sobre almacenamiento de objetos.

●**Recuperación Aumentada por Generación (RAG):** Un modelo de lenguaje complementa su ventana de contexto con fragmentos de texto recuperados dinámicamente desde un almacén externo, en lugar de depender únicamente de lo memorizado durante el entrenamiento.

●**Búsqueda por similitud vectorial:** El mecanismo que hace posible el punto anterior. Cada fragmento de texto (o imagen, o entidad) se transforma en un vector de embedding de alta dimensionalidad, recuperar información "semánticamente similar" a una consulta se convierte en un problema de **búsqueda del vecino más cercano aproximado (ANN)** sobre ese espacio vectorial, típicamente resuelto con estructuras de índice como HNSW (grafos de mundo pequeño navegables) en lugar de los B-trees tradicionales de un RDBMS. Este es precisamente el problema que sistemas como Milvus abordan como motores de base de datos de propósito específico.

Referencias

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>

Date, C. J. (2004). *An introduction to database systems* (8.^a ed.). Pearson/Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.^a ed.). Pearson.

Ramakrishnan, R., & Gehrke, J. (2003). *Database management systems* (3.^a ed.). McGraw-Hill.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7.^a ed.). McGraw-Hill.